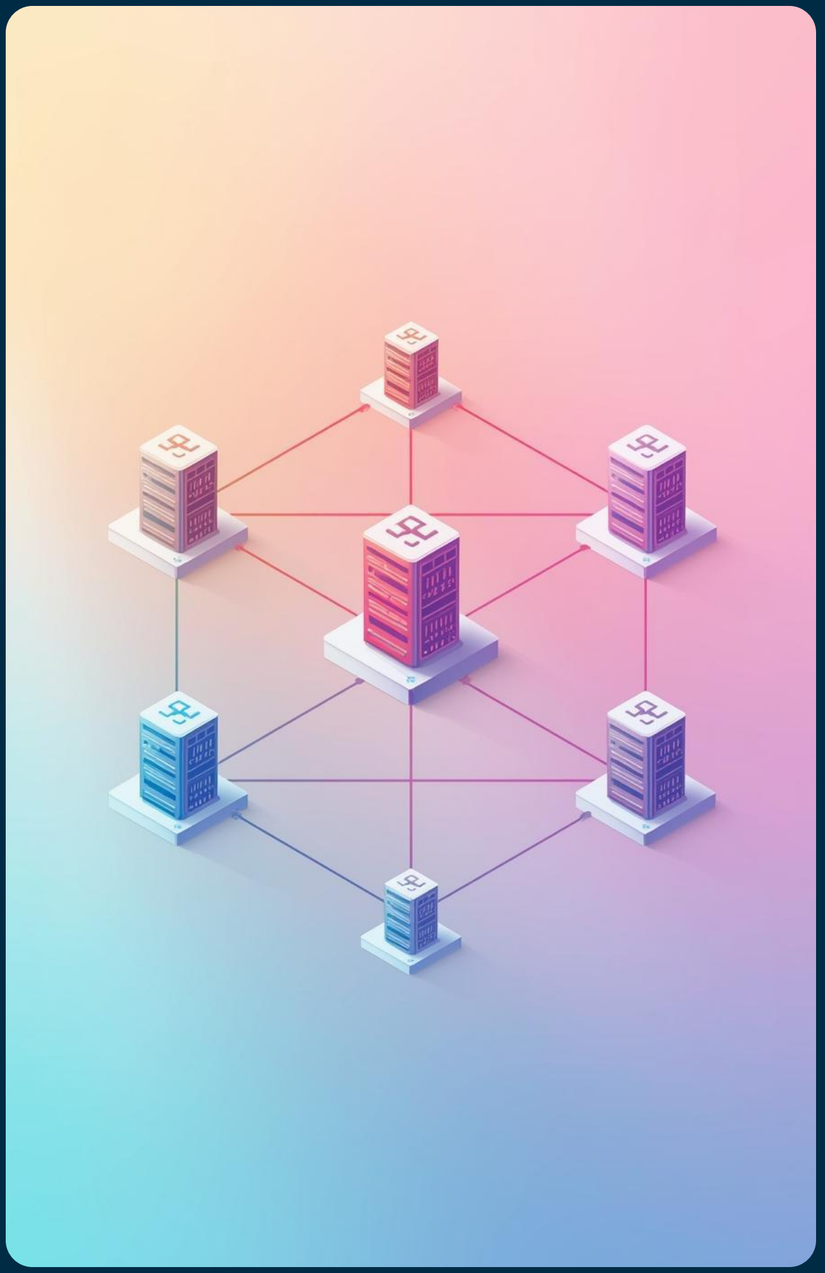
DOCENTE	**Fabio ABBA’**
UNITA’ FORMATIVA	**Competenze informatiche/digitali**
ARGOMENTO	**RESTful API**

Creare un'API REST

Docente: Fabio Abbà

TC-Web
Technology Consulting

Intervento realizzato da
ITS TECNOLOGIE DELL'INFORMAZIONE E DELLA COMUNICAZIONE



Definizioni

Per API (Application Programming Interface) si intende un insieme di regole e convenzioni che permette a due software di comunicare tra loro: è la lingua comune che entrambi parlano e capiscono, il «menu» del ristorante che ci dice cosa c'è a disposizione.

Conoscendo la struttura delle API a disposizione, il client invia una richiesta che potrà essere compresa dal server, senza bisogno di conoscerne il suo funzionamento interno.

Facciamo l'esempio dell'API di un servizio meteo: essa sarà costituita dall'insieme di endpoint (URI) correlati che espongono le risorse fornite da un servizio:

- https://weather.api.com/current
- https://weather.api.com/forecast?days=3
- https://weather.api.com/air-quality



Pianificazione di un’API

Per creare un'API è utile delinearne preventivamente la struttura, ricordando che l'architettura REST prevede che gli endpoit da chiamare rispecchino le risorse (oggetti) a disposizione e facciano uso dei verbi HTTP. Qualche convenzione di nomenclatura sarà di aiuto.

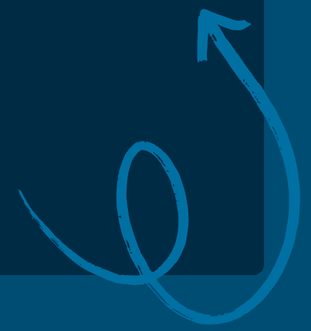


Convenzioni di Nomenclatura

Chiarezza e coerenza negli endpoint

Per cominciare, è importante che si seguano delle convenzioni uniformi in tutto l'applicativo; questo rende l'esperienza più prevedibile ed intuitiva per l'utente ma anche per lo sviluppatore che deve metterci mano (specialmente se non è stato lui a scrivere quel codice!).

Per esempio: se abbiamo /users/{id}/orders per gli ordini di un utente, dovremmo seguire lo stesso pattern per altre relazioni, come /users/{id}/addresses o /products/{id}/reviews

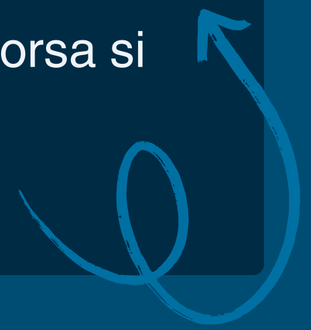


Convenzioni di Nomenclatura

Oppure, evitare di usare nomi diversi per lo stesso campo (created_at e creation_date), oppure case diversi (created_at e createdAt).

Voi mi direte: basta un po' di buon senso!
E io vi dico che ne sono provviste molte meno persone di quelle che pensate. Ma sorvoliamo...

Per le collezioni di dati si usa il plurale (es. /products, /users); per identificare la singola risorsa si usa comunemente il suo id usato come path variable (es. /products/123, /users/456).

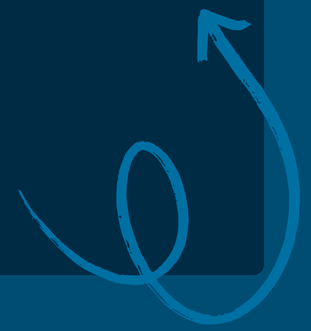


Convenzioni di Nomenclatura

In questo modo si rende chiaro che si agisce andando a cercare un dettaglio all'interno della collezione.

Beninteso: la struttura può annidarsi ulteriormente, ad es.
/users/456/orders/112233 → sto andando a cercare tra gli ordini dell'utente 456 quello con il numero 112233.

La gerarchia del path di solito rispecchia la relazione tra le risorse.



Convenzioni di Nomenclatura

Altrettanto importante ai fini della chiarezza è l'uso dei verbi HTTP:

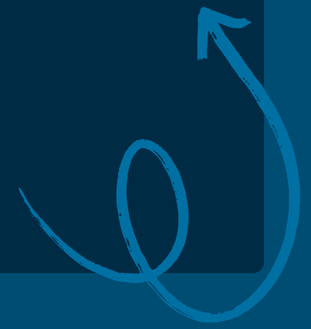
- GET /products -> ritorna la lista di utenti
- GET /products/123 -> ritorna un prodotto specifico
- POST /products -> aggiunge un nuovo prodotto (non è specificato l'id, solitamente è aggiunto dal db)
- PUT /products/123 -> aggiorna il prodotto con id 123
- DELETE /products/123 -> elimina il prodotto con id 123



Convenzioni di Nomenclatura

Naturalmente le chiamate GET possono prevedere dei query params (*?nomeParametro=valore*) per raffinare la ricerca (comunemente filtraggio, ordinamento o paginazione).

- GET /users?role=admin
- GET /products?page=2&limit=20
- GET /products?sort=field&order=asc



Versionamento

Infine, non è una cattiva idea usare una strategia di versionamento.

Se non abbiamo fatto un lavoro osceno destinato ad estinguersi da sè, prima o poi dovremo aggiornare qualcosa nel nostro applicativo.

Finchè si aggiungono cose, di solito non si creano grandi problemi. Ma nel caso in cui ci fossero eliminazioni di endpoint, campi o comunque modifiche sostanziali alle nostre API, bisognerà prevedere un meccanismo per creare una finestra di tempo che consenta agli utenti di aggiornare i loro sistemi.

(E, detto tra noi, non è male anche per avere un backup nel caso in cui la nuova versione fosse il lavoro osceno di cui sopra...)



Versionamento

L'approccio più popolare e diffuso è includere la versione nell'URL:

/v1/users, /v2/users, /v3/users

La versione è tipicamente subito dopo il dominio, prima delle risorse.

È esplicito, immediatamente visibile, e facile da implementare e capire.



Riepilogando

Quindi, mettiamo insieme:

- nomenclatura uniforme
- gerarchia nel path
- uso verbi HTTP
- versionamento

Usiamo tutto questo per definire l’API “Catalogo” dell'applicazione "libreria" che vogliamo creare.



Verifichiamo



```
GET /api/v1/catalogo         -> restituisce l'intero catalogo dei libri
GET /api/v1/generi           -> restituisce l'elenco dei generi dei libri
GET /api/v1/catalogo/search -> specificando il titolo come query param, restituisce un array di libri
                               il cui titolo contiene la parola fornita
GET /api/v1/catalogo/:id     -> restituisce il dettaglio del libro cercato (se presente, se no 404)

POST /api/v1/catalogo        -> fornendo un json valido, crea un nuovo libro
PUT  /api/v1/catalogo/:id    -> fornendo un json valido, aggiorna il libro specificato
DELETE /api/v1/catalogo/:id -> rimuove il libro dal catalogo (se presente)
```

Ora a voi!

Create le API relative alla gestione dei clienti e degli ordini ad essi correlati.

Per comodità, assumiamo che i clienti siano allo stesso livello gerarchico dei libri.

E poi inizieremo a scrivere il codice necessario sul nostro server node.js.





Grazie per l'attenzione!

Docente: Fabio Abbà

fabio.abba@tc-web.it

Via Giovanni Schiaparelli 14,
10148 – Torino (TO)
E-mail: info@tc-web.it
Tel: 011 0466685